



Universidad Tecnológica de Bolívar

UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

INTELIGENCIA ARTIFICIAL

***Agente Inteligente de Ciberseguridad***

*Mauro Alonso González Figueroa, T00067622*

*Juan Jose Jiménez Guardo, T00068278*

*Revisado Por*

*Edwin Alexander Puertas Del Castillo*

*17 de octubre de 2025*

# Índice

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduccion</b>                                 | <b>3</b>  |
| 1.1      | Caso de Estudio . . . . .                           | 3         |
| 1.2      | Resumen Ejecutivo . . . . .                         | 3         |
| <b>2</b> | <b>Modulo <i>PEAS</i></b>                           | <b>4</b>  |
| 2.1      | Performance (Medición del Desempeño) . . . . .      | 4         |
| 2.2      | Environment (Entorno) . . . . .                     | 4         |
| 2.3      | Actuators (Actuadores) . . . . .                    | 5         |
| 2.4      | Sensors (Sensores) . . . . .                        | 5         |
| <b>3</b> | <b>Descripción del Agente</b>                       | <b>5</b>  |
| 3.1      | Plataforma Utilizada . . . . .                      | 6         |
| 3.2      | Tipo de Agente . . . . .                            | 6         |
| 3.3      | Justificación de Elección de Entorno . . . . .      | 7         |
| <b>4</b> | <b>Diseño del Agente</b>                            | <b>7</b>  |
| 4.1      | Arquitectura General . . . . .                      | 7         |
| 4.2      | Descripción del Comportamiento del Agente . . . . . | 7         |
| 4.3      | Flujo Percepción, Razonamiento, Acción . . . . .    | 8         |
| <b>5</b> | <b>Representación del Conocimiento</b>              | <b>8</b>  |
| 5.1      | Estructuras de Datos y Reglas . . . . .             | 9         |
| 5.2      | Base de Conocimiento . . . . .                      | 9         |
| 5.3      | Consideraciones para Futuras Mejoras . . . . .      | 9         |
| <b>6</b> | <b>Implementación Técnica</b>                       | <b>10</b> |
| 6.1      | Integración de Componentes . . . . .                | 10        |
| 6.2      | Flujo de Trabajo del Agente . . . . .               | 10        |
| 6.3      | Actualización Dinámica de Riesgo . . . . .          | 11        |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 6.4      | Motor de Razonamiento . . . . .        | 11        |
| <b>7</b> | <b>Evaluación</b>                      | <b>12</b> |
| 7.1      | Métricas de Éxito . . . . .            | 12        |
| 7.2      | Resultados Obtenidos . . . . .         | 13        |
| 7.3      | Casos de Prueba . . . . .              | 13        |
| 7.4      | Limitaciones y Observaciones . . . . . | 14        |
| <b>8</b> | <b>Conclusiones y Mejoras Futuras</b>  | <b>14</b> |
|          | <b>Anexos</b>                          | <b>16</b> |

# 1. Introduccion

## 1.1. Caso de Estudio

La defensa cibernética es un conjunto de estrategias, tecnologías y procesos diseñados para proteger los sistemas informáticos, redes y datos contra amenazas cibernéticas, como ataques de malware, piratería informática, robo de datos y otros riesgos de seguridad. Estas medidas de defensa buscan prevenir, detectar, responder y recuperarse de ataques cibernéticos con el fin de garantizar la confidencialidad, integridad y disponibilidad de la información y los recursos digitales.

## 1.2. Resumen Ejecutivo

En el contexto actual de la transformación digital, la ciberseguridad se ha convertido en un pilar fundamental para la protección de los sistemas de información. Las amenazas informáticas evolucionan constantemente, lo que exige el desarrollo de soluciones inteligentes capaces de detectar, analizar y responder a incidentes de seguridad de manera eficiente.

Este proyecto presenta el desarrollo inicial de un **agente inteligente de ciberseguridad**, implementado utilizando tecnologías modernas como **Python**, el framework web **Django** y el sistema de gestión de bases de datos **PostgreSQL**. Aunque el agente aún se encuentra en una etapa temprana y su funcionalidad es limitada, se ha logrado establecer una arquitectura básica que permite la integración y gestión de datos relevantes para la detección de amenazas.

El sistema está diseñado para operar como un agente autónomo dentro de una infraestructura web, recolectando información sobre eventos y posibles vulnerabilidades. La elección de Django facilita la construcción de una interfaz administrativa y la gestión eficiente de los modelos de datos, mientras que PostgreSQL proporciona robustez y escalabilidad en el almacenamiento de información.

Cabe resaltar que, si bien el agente aún no implementa técnicas avanzadas de inteligencia artificial, se han sentado las bases para futuras mejoras, incluyendo la incorporación de algoritmos de aprendizaje automático y módulos de análisis automatizado. El objetivo

principal de este informe es documentar el proceso de diseño y desarrollo, así como los retos enfrentados y las oportunidades de mejora identificadas en el camino.

## 2. Modulo *PEAS*

El diseño del agente inteligente se describe formalmente mediante el modelo **PEAS** (Performance, Environment, Actuators, Sensors), el cual permite definir de manera estructurada los elementos fundamentales de su operación.

### 2.1. Performance (Medición del Desempeño)

Las métricas utilizadas para evaluar el desempeño del agente incluyen:

- Tiempo Medio de Respuesta (MTTR).
- Tiempo Medio Entre Fallos (MTBF).
- Efectividad de contención ante incidentes.
- Porcentaje de ataques prevenidos.

### 2.2. Environment (Entorno)

El agente opera en:

- Infraestructura de red corporativa.
- Servidores críticos y servicios en la nube.
- Dispositivos IoT y BYOD conectados a la red.
- Factores humanos y organizacionales, incluyendo políticas de seguridad, niveles de autorización y comportamiento del usuario.

## 2.3. Actuators (Actuadores)

Los actuadores del sistema incluyen:

- Bloqueo automático de tráfico malicioso.
- Aislamiento de dispositivos comprometidos.
- Módulo SOAR para respuesta orquestada (bloqueo, escalamiento, notificación).
- Generación de reportes ejecutivos y dashboards en tiempo real.
- Aplicación de parches y restauración de servicios.

## 2.4. Sensors (Sensores)

El agente integra sensores especializados para la detección de amenazas:

- Sensores de red para tráfico y anomalías.
- Monitores de integridad en sistemas y archivos.
- Antivirus y detección basada en firmas.
- Sensores basados en inteligencia de amenazas externa.
- Sensores de comportamiento de usuario (UEBA).

# 3. Descripción del Agente

El agente desarrollado en este proyecto tiene como objetivo principal la detección y gestión de riesgos cibernéticos en una infraestructura web. Su diseño se basa en la arquitectura modular de **Python** y el framework **Django**, permitiendo la integración de diferentes módulos de análisis y la administración eficiente de los datos recolectados.

El agente opera de manera autónoma, procesando solicitudes HTTP y aplicando un ciclo de percepción, razonamiento y acción. Para cada solicitud, el sistema realiza análisis de

geolocalización, reputación de IP (VirusTotal) y evaluación heurística mediante un motor de razonamiento centralizado. Los resultados de estos análisis se almacenan en una base de datos **PostgreSQL**, permitiendo la actualización dinámica de los indicadores de riesgo tanto por país como por dirección IP.

La arquitectura propuesta facilita la escalabilidad y la incorporación de nuevos módulos de análisis en el futuro, como algoritmos de inteligencia artificial o integración con plataformas externas de ciberseguridad.

### 3.1. Plataforma Utilizada

El agente se implementó utilizando las siguientes tecnologías:

- **Python 3.11**: Lenguaje principal de desarrollo.
- **Django 4.x**: Framework web para la gestión de modelos, vistas y lógica de negocio.
- **PostgreSQL**: Sistema de gestión de bases de datos relacional, utilizado para almacenar los registros de riesgo y eventos.
- **VirusTotal API**: Servicio externo para la evaluación de reputación de direcciones IP.
- **GeoLite2**: Base de datos de geolocalización para el análisis de ubicación de las IPs.

### 3.2. Tipo de Agente

El agente desarrollado puede clasificarse como un **agente híbrido**, ya que combina procesamiento lógico, heurístico y la integración de servicios externos basados en inteligencia artificial. Aunque la base del sistema se apoya en reglas y estructuras de datos tradicionales, se contempla la incorporación de módulos de razonamiento avanzado y análisis automatizado en futuras versiones.

### 3.3. Justificación de Elección de Entorno

La elección de **Python** y **Django** como entorno de desarrollo se justifica por su amplia adopción en el ámbito de la ciberseguridad, la facilidad para construir aplicaciones modulares y la disponibilidad de librerías especializadas para el análisis de datos y la integración con APIs externas. Django proporciona una estructura robusta para la gestión de modelos y vistas, mientras que Python facilita la implementación de lógica compleja y el manejo eficiente de flujos de datos.

El uso de **PostgreSQL** como base de datos garantiza escalabilidad y confiabilidad en el almacenamiento de registros de riesgo, y la integración con servicios como **VirusTotal** y **GeoLite2** permite enriquecer el análisis de amenazas con información actualizada y relevante.

## 4. Diseño del Agente

### 4.1. Arquitectura General

El agente está estructurado en módulos independientes que interactúan a través de un flujo centralizado de procesamiento de solicitudes. La arquitectura se basa en el patrón percepción → razonamiento → acción, donde cada solicitud HTTP desencadena una serie de análisis y decisiones automatizadas.

La arquitectura completa del agente se encuentra aquí:

<https://github.com/MauroGonzalez51/cibersecurity-agent/blob/master/docs/pipeline.png>

### 4.2. Descripción del Comportamiento del Agente

El ciclo de funcionamiento del agente se compone de las siguientes etapas:

1. **Percepción:** El agente recibe una solicitud HTTP y extrae información relevante, como la dirección IP de origen, el país asociado y otros metadatos.
2. **Análisis:** Se ejecutan módulos especializados:



- **GeoIP:** Determina la ubicación geográfica de la IP y actualiza los contadores de riesgo por país.
  - **VirusTotal:** Consulta la reputación de la IP en bases de datos externas.
  - **IA/Heurística:** Evalúa el contexto y posibles amenazas mediante reglas y ponderaciones.
3. **Razonamiento:** El motor de razonamiento centraliza los resultados de los análisis, pondera los riesgos y decide la acción a tomar (permitir o bloquear la solicitud).
  4. **Acción:** Según la decisión, el agente actualiza los registros en la base de datos y retorna una respuesta al usuario o sistema externo.

### 4.3. Flujo Percepción, Razonamiento, Acción

El flujo completo se puede resumir como:

- **Percepción:** Recolección de datos de la solicitud y del entorno.
- **Razonamiento:** Evaluación de riesgos y amenazas mediante la integración de múltiples fuentes de información.
- **Acción:** Ejecución de la decisión (bloqueo, registro, notificación).

## 5. Representación del Conocimiento

La representación del conocimiento en el agente se basa principalmente en estructuras de datos y reglas condicionales implementadas en Python. El sistema no utiliza lógica de primer orden (FOL) ni memoria conversacional avanzada, pero sí integra mecanismos para almacenar y actualizar información relevante sobre el contexto de cada solicitud.

## 5.1. Estructuras de Datos y Reglas

El conocimiento sobre el entorno y las amenazas se almacena en modelos de base de datos relacional (`CountryRiskRecord`, `IpRiskRecord`), donde se registran atributos como la cantidad de solicitudes, el número de eventos maliciosos y los puntajes de riesgo asociados a cada país o dirección IP. Estos registros permiten al agente mantener un historial dinámico y actualizado del comportamiento observado.

Las decisiones se toman mediante un motor de razonamiento que aplica reglas heurísticas y ponderaciones sobre los resultados de los módulos de análisis (GeoIP, VirusTotal, IA). Por ejemplo, si el puntaje de riesgo acumulado supera un umbral predefinido, la solicitud es bloqueada automáticamente.

## 5.2. Base de Conocimiento

La base de conocimiento del agente está compuesta por los datos almacenados en la base de datos PostgreSQL, que incluyen:

- Estadísticas de riesgo por país (`CountryRiskRecord`).
- Estadísticas de riesgo por dirección IP (`IpRiskRecord`).
- Resultados de análisis de reputación y geolocalización.

Esta información es consultada y actualizada en tiempo real durante el procesamiento de cada solicitud, permitiendo que el agente adapte su comportamiento en función de la experiencia acumulada.

## 5.3. Consideraciones para Futuras Mejoras

Aunque actualmente el agente no implementa lógica FOL ni mecanismos de memoria avanzada, la arquitectura modular permite la incorporación futura de técnicas como *Retrieval-Augmented Generation* (RAG), integración con bases de conocimiento externas o el uso de modelos de lenguaje para razonamiento más sofisticado.

## 6. Implementación Técnica

### 6.1. Integración de Componentes

El agente está construido sobre una arquitectura modular que facilita la integración de análisis de geolocalización, reputación de IP y razonamiento heurístico. Cada módulo se comunica a través de un flujo centralizado, gestionado por Django, que orquesta la recepción de solicitudes, el procesamiento de datos y la toma de decisiones.

La base de datos relacional almacena dinámicamente los indicadores de riesgo por país e IP, permitiendo que el agente adapte su comportamiento en función de la experiencia acumulada. La consulta a servicios externos, como VirusTotal y GeoLite2, enriquece el análisis sin comprometer la eficiencia del sistema.

### 6.2. Flujo de Trabajo del Agente

El ciclo principal del agente sigue el patrón percepción  $\rightarrow$  razonamiento  $\rightarrow$  acción. El siguiente fragmento ilustra el flujo de procesamiento de una solicitud:

```
1 @csrf_exempt
2 def agent(request: HttpRequest):
3     callback_url = extract_callback_url(request=request)
4     if not callback_url:
5         return JsonResponse(HttpAgentDecision(
6             decision="BLOCK",
7             risk_score=100,
8             threats=["missing_callback_url"],
9             callback_url=callback_url,
10            context=None,
11        ))
12     context = build_request_context(request=request, callback_url=
callback_url)
13     _ia, _vt, _geo = ia(context=context), vt(context=context), geo(context
=context)
14     decision = reasoning(context=context, vt=_vt, ia=_ia, geo=_geo)
```

```
15     return JsonResponse(decision.model_dump_json(indent=4))
```

### 6.3. Actualización Dinámica de Riesgo

El agente actualiza automáticamente los registros de riesgo asociados a cada país e IP. Por ejemplo, al recibir una solicitud, incrementa el contador de solicitudes y, si corresponde, el de eventos maliciosos:

```
1 with transaction.atomic():
2     obj, created = CountryRiskRecord.objects.select_for_update().
3     get_or_create(
4         country_code=country_code,
5         defaults=dict(total_requests=1, malicious_requests=0, risk_score
6         =0.0)
7     )
8     if not created:
9         obj.total_requests = models.F("total_requests") + 1
10        obj.save()
```

### 6.4. Motor de Razonamiento

El motor de razonamiento centraliza la lógica de decisión, ponderando los resultados de los análisis y penalizando la falta de información relevante. Si el puntaje de riesgo supera un umbral, la solicitud es bloqueada automáticamente:

```
1 def reasoning(context, ia, vt, geo):
2     risk_score = 0.0
3     threats = []
4     if ia is None:
5         risk_score += 40
6         threats.append("ia_unavailable")
7     # ... (logica para vt y geo)
8     if risk_score >= 70 or "ia_unavailable" in threats:
9         decision = "BLOCK"
```

```

10     else:
11         decision = "ALLOW"
12     return HttpAgentDecision(
13         decision=decision,
14         error="",
15         risk_score=risk_score,
16         threats=threats,
17         callback_url=context.callback_url,
18     )

```

## 7. Evaluación

Es importante señalar que, en su estado actual, el sistema implementado cumple únicamente con las funciones básicas de un agente: recibe una URL, analiza diversos datos asociados al usuario y al request, y toma una decisión automatizada de permitir o bloquear la solicitud. Aunque la arquitectura está pensada para ser extensible y modular, aún no incorpora capacidades avanzadas de aprendizaje, adaptación o interacción autónoma propias de agentes inteligentes más sofisticados. Por tanto, los resultados aquí presentados deben interpretarse como una validación de la base tecnológica y del flujo de decisión, más que como una evaluación de un agente plenamente autónomo.

### 7.1. Métricas de Éxito

La evaluación del sistema se realizó considerando las siguientes métricas:

- **Porcentaje de solicitudes correctamente clasificadas:** Proporción de solicitudes permitidas o bloqueadas de acuerdo con su naturaleza (benigna o maliciosa).
- **Latencia promedio de respuesta:** Tiempo medio desde la recepción de una solicitud hasta la emisión de una decisión.

- **Cobertura de análisis:** Porcentaje de solicitudes para las cuales se ejecutaron todos los módulos de análisis (GeoIP, VirusTotal, IA).
- **Tasa de falsos positivos/negativos:** Proporción de solicitudes benignas bloqueadas y maliciosas permitidas, respectivamente.

## 7.2. Resultados Obtenidos

Debido a limitaciones de tiempo y recursos, la evaluación se realizó sobre un conjunto reducido de pruebas simuladas. Los resultados obtenidos fueron los siguientes:

- El sistema procesó un total de **30 solicitudes** en un entorno de prueba controlado.
- El **porcentaje de solicitudes correctamente clasificadas** fue del **70 %**, con una tasa de falsos positivos del 17 % y falsos negativos del 13 %.
- La **latencia promedio de respuesta** fue de aproximadamente **410 ms** por solicitud, incluyendo el tiempo de consulta a servicios externos.
- La **cobertura de análisis** alcanzó el 90 %, siendo los fallos atribuibles principalmente a la falta de conectividad con servicios externos en momentos puntuales.

Estos resultados muestran que el sistema es capaz de integrar múltiples fuentes de información y tomar decisiones razonadas en tiempos adecuados para un entorno web, aunque existen oportunidades claras de mejora.

## 7.3. Casos de Prueba

Se diseñaron y ejecutaron los siguientes casos de prueba para validar el comportamiento del sistema:

1. **Solicitud desde IP benigna:** El sistema permitió el acceso y actualizó los contadores de riesgo de forma adecuada.

2. **Solicitud desde IP con reputación negativa en VirusTotal:** El sistema bloqueó la solicitud y registró el evento como malicioso.
3. **Solicitud desde país con alto riesgo acumulado:** El sistema incrementó el puntaje de riesgo y, al superar el umbral, bloqueó la solicitud.
4. **Fallo en el análisis de algún módulo (por ejemplo, VirusTotal inaccesible):** El sistema penalizó la falta de información y, en caso de riesgo acumulado, optó por bloquear la solicitud.

## 7.4. Limitaciones y Observaciones

La evaluación se realizó en un entorno controlado y con datos simulados, por lo que los resultados pueden variar en un entorno de producción real. Se recomienda realizar pruebas adicionales con tráfico real y ajustar los umbrales de decisión según el contexto operativo específico.

## 8. Conclusiones y Mejoras Futuras

El desarrollo de este sistema representa una aproximación inicial a la construcción de un agente inteligente de ciberseguridad. Si bien el agente implementado cumple únicamente con funciones básicas de análisis y decisión sobre solicitudes HTTP, la arquitectura modular y la integración con servicios externos sientan las bases para futuras extensiones.

Entre los principales logros se destaca la capacidad del sistema para recolectar información relevante de cada solicitud, consultar fuentes externas de reputación y geolocalización, y tomar decisiones automatizadas de bloqueo o permiso. Además, el uso de una base de datos relacional permite la actualización dinámica de los indicadores de riesgo, tanto por país como por dirección IP.

Sin embargo, el sistema actual presenta limitaciones importantes. No incorpora mecanismos de aprendizaje automático, memoria avanzada ni razonamiento autónomo, características propias de agentes inteligentes más sofisticados. La evaluación se realizó en un entorno

controlado y con datos simulados, por lo que se requiere una validación más exhaustiva en escenarios reales.

Como mejoras futuras se proponen:

- Incorporar módulos de aprendizaje automático para la detección proactiva de amenazas.
- Implementar mecanismos de memoria y razonamiento avanzado, como *Retrieval-Augmented Generation* (RAG) o integración con bases de conocimiento externas.
- Ampliar la cobertura de análisis, integrando nuevas fuentes de información y servicios de inteligencia de amenazas.
- Realizar pruebas en entornos de producción y ajustar los umbrales de decisión según el contexto operativo.
- Desarrollar una interfaz de monitoreo y visualización para facilitar la gestión y auditoría de eventos.

En conclusión, el trabajo realizado constituye una base sólida sobre la cual se pueden construir agentes de ciberseguridad más avanzados y adaptativos, capaces de responder de manera eficiente a los desafíos de un entorno digital en constante evolución.



# Anexos

## Código Fuente

El código fuente completo del agente se encuentra disponible en el repositorio de GitHub:  
<https://github.com/MauroGonzalez51/cibersecurity-agent>

## Variables de Entorno Requeridas

Para el correcto funcionamiento del sistema, es necesario definir las siguientes variables de entorno en un archivo `.env` en la raíz del proyecto:

- `POSTGRES_USER`: Usuario de la base de datos PostgreSQL.
- `POSTGRES_PASSWORD`: Contraseña de la base de datos PostgreSQL.
- `POSTGRES_HOST`: Host de la base de datos PostgreSQL.
- `POSTGRES_PORT`: Puerto de la base de datos PostgreSQL (por defecto, 5432).
- `POSTGRES_DB`: Nombre de la base de datos PostgreSQL.
- `OPENROUTER_API_KEY`: Clave de API para el servicio OpenRouter.
- `VIRUSTOTAL_API_KEY`: Clave de API para el servicio VirusTotal.
- `GEOIP_DATABASE_PATH`: Ruta al archivo de base de datos GeoLite2-City.mmdb (Opcional).